

Most Important Basic ML Viva Questions & Answers

Essential Questions Every Student Must Know

SECTION 1: FUNDAMENTAL CONCEPTS

Q1: What is Machine Learning?

Answer: Machine Learning is a subset of Artificial Intelligence that enables computers to learn from data without being explicitly programmed.

Three main types:

1. **Supervised Learning:** Learning from labeled data (we know correct answers)
2. **Unsupervised Learning:** Finding patterns in unlabeled data
3. **Reinforcement Learning:** Learning through trial and error with rewards

Example from our project:

- Supervised: Predicting discount (we have labels: 0 or 1)
 - Unsupervised: K-Means clustering (finding customer groups)
-

Q2: What is the difference between Classification and Regression?

Answer:

Classification:

- **Predicts categories/classes** (discrete values)
- Output: Yes/No, Cat/Dog/Bird, 0/1
- Examples: Email spam detection, disease diagnosis
- **Our project:** Discount Applied (0 = No, 1 = Yes)

Regression:

- **Predicts continuous numbers**
- Output: Price, temperature, age
- Examples: House price prediction, stock price
- **Our project:** Total Spent (any value from \$0 to \$400+)

Key Difference:

- Classification: "Which category?"
 - Regression: "How much?"
-

Q3: What is overfitting and underfitting?

Answer:

Overfitting:

- Model learns training data **TOO WELL** (including noise)
- **High training accuracy, low test accuracy**
- Like a student who memorizes answers but can't solve new problems
- **Example from our project:** KNN (76% train, 51% test = 25% gap)

Underfitting:

- Model is **TOO SIMPLE** to capture patterns
- **Both training and test accuracy are low**
- Like a student who doesn't study enough
- **Example:** If we used only 1 feature

Good Fit:

- Balanced performance on both train and test
- **Example from our project:** Logistic Regression (50.8% train, 50% test)

Visual:

Underfitting: ----- (straight line for curved data)

Good Fit: ~~~~~~ (smooth curve fitting data)

Overfitting: ~~~~~~ (zigzag through every point)

Q4: What is train-test split? Why do we need it?

Answer:

Train-Test Split: Dividing dataset into two parts:

- **Training Set (80%):** Used to teach the model
- **Test Set (20%):** Used to evaluate the model

Why we need it:

1. **Prevents Overfitting Detection:** Can't evaluate on same data used for training
2. **Simulates Real World:** Test set represents new, unseen data
3. **Honest Performance:** Shows how model works on future data

Example:

Total Data: 12,575 transactions

└─ Train: 10,060 (80%) → Model learns here

└─ Test: 2,515 (20%) → Model tested here

Key Rule: NEVER use test data for training!

Our project: 80-20 stratified split to maintain class balance

Q5: What is a confusion matrix?

Answer:

Confusion Matrix: A table showing model's predictions vs. actual values

Structure:

		PREDICTED	
		No	Yes
ACTUAL	No	TN	FP
	Yes	FN	TP

Four Components:

- 1. **TP (True Positive):** Correctly predicted YES (predicted discount, actually discount)
- 2. **TN (True Negative):** Correctly predicted NO (predicted no discount, actually no discount)
- 3. **FP (False Positive):** Incorrectly predicted YES (predicted discount, but no discount) - Type I Error
- 4. **FN (False Negative):** Incorrectly predicted NO (predicted no discount, but actually discount) - Type II Error

Example:

		Predicted		
		No Disc	Discount	
Actual	No	1500	200	← 200 False Alarms
	Yes	400	415	← 400 Missed Cases

Why Important: Shows WHERE the model makes mistakes, not just how many

SECTION 2: EVALUATION METRICS

Q6: What is Accuracy? When is it misleading?

Answer:

Accuracy Formula:

Accuracy = (TP + TN) / Total

= Correct Predictions / All Predictions

Example: If model makes 80 correct predictions out of 100: Accuracy = 80/100 = 80%

When Misleading - Imbalanced Data:

Example:

- 95 samples of Class A (No Discount)

- 5 samples of Class B (Discount)

Dummy Model: Always predict "No Discount"

- Accuracy = 95% (looks great!)
- But catches ZERO actual discounts (useless!)

Our Project:

- 66% data is "No Discount"
- A model always predicting "No Discount" gets 66% accuracy
- Decision Tree: 63.62% accuracy (only slightly better than dummy!)

When to Use: Balanced datasets only

Q7: What is Precision and Recall? When to use which?

Answer:

Precision:

Precision = $TP / (TP + FP)$
= "Of all YES predictions, how many were correct?"

Focus: Minimize false alarms

Recall (Sensitivity):

Recall = $TP / (TP + FN)$
= "Of all actual YES cases, how many did we catch?"

Focus: Minimize missed cases

Example - Disease Detection:

High Precision Needed:

- Expensive treatment
- Want to be SURE patient has disease before treating
- Okay to miss some cases

High Recall Needed:

- Deadly disease
- Must catch ALL patients even if some false alarms
- Better safe than sorry

Our Project:

- **Logistic Regression:** Recall = 48.82% (catches more discounts)
- **Decision Tree:** Recall = 8.89% (misses most discounts)

Trade-off: Usually can't maximize both simultaneously

Q8: What is AUC-ROC curve?

Answer:

ROC (Receiver Operating Characteristic) Curve:

- Graph showing model performance at ALL classification thresholds
- X-axis: False Positive Rate (FPR)
- Y-axis: True Positive Rate (TPR = Recall)

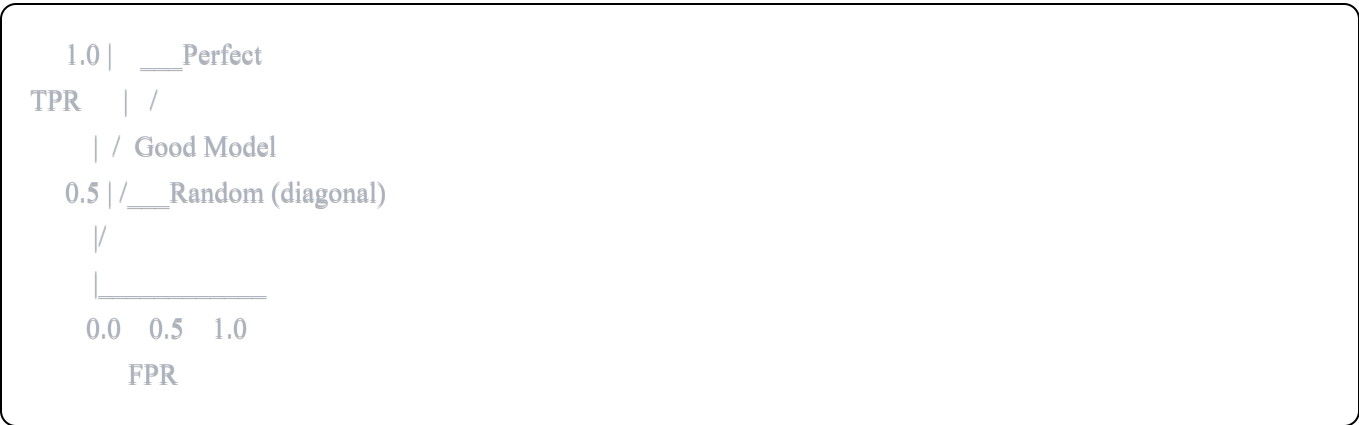
AUC (Area Under Curve):

- Single number summarizing ROC curve
- **Range: 0 to 1**

Interpretation:

- **AUC = 1.0:** Perfect classifier
- **AUC = 0.9-1.0:** Excellent
- **AUC = 0.8-0.9:** Good
- **AUC = 0.7-0.8:** Fair
- **AUC = 0.5:** Random guessing (diagonal line)
- **AUC < 0.5:** Worse than random

Visual:



Our Project:

- All AUC scores < 0.5 (KNN: 0.49, DT: 0.49, LR: 0.48, NN: 0.47)
 - **Means:** Models perform worse than random guessing!
 - **Reason:** Features have no predictive power
-

SECTION 3: DATA PREPROCESSING

Q9: Why do we need feature scaling?

Answer:

Problem Without Scaling: Features have different ranges:

- Age: 20-80
- Income: 20,000-200,000
- Height: 150-190 cm

Algorithms like KNN and Neural Networks treat larger numbers as more important!

Example:

Person A: Age=25, Income=50,000
Person B: Age=30, Income=51,000

Distance = $\sqrt{[(30-25)^2 + (51000-50000)^2]}$
 = $\sqrt{[25 + 1,000,000]}$
 $\approx 1,000$

Income dominates! Age difference (5 years) is ignored.

After Scaling: All features contribute equally

Two Methods:

1. StandardScaler (Z-score normalization):

$$z = (x - \text{mean}) / \text{std_deviation}$$

- Result: Mean = 0, Std = 1
- **Use when:** Data is normally distributed
- **Our choice** ✓

2. MinMaxScaler:

$$x_scaled = (x - \text{min}) / (\text{max} - \text{min})$$

- Result: Range [0, 1]
- **Use when:** Need specific range

Algorithms that NEED scaling:

- KNN, Neural Networks, SVM, Logistic Regression

Algorithms that DON'T need scaling:

- Decision Trees, Random Forest

Q10: What is SMOTE? Why did you use it?

Answer:

SMOTE: Synthetic Minority Over-sampling Technique

Problem - Class Imbalance:



Models learn: "Just predict No Discount always" → 66% accuracy!

How SMOTE Works:

1. Take a minority sample
2. Find its K nearest neighbors (same class)
3. Create new synthetic sample between them
4. Repeat until classes balanced

Example:



Results in Our Project:



CRITICAL: Apply SMOTE only to training data (not test data!)

Alternatives:

- Oversampling: Copy minority samples (causes overfitting)
- Undersampling: Remove majority samples (loses data)
- SMOTE: Best of both (creates new synthetic samples)

Q11: What is Label Encoding vs One-Hot Encoding?

Answer:

Problem: ML algorithms need numbers, not text

Label Encoding: Assigns each category a number



Advantages:

- Simple, one column stays one column
- Works well with tree-based models

Disadvantage:

- Creates false ordering: Model thinks $2 > 1 > 0$
- May think Patisserie (2) is "greater" than Food (0)

One-Hot Encoding: Creates binary column for each category

Category:	Food	Beverage	Patisserie
Food:	[1,	0,	0]
Beverage:	[0,	1,	0]
Patisserie:	[0,	0,	1]

Advantages:

- No false ordering
- Each category independent

Disadvantage:

- Many columns (3 categories $\rightarrow$ 3 columns)
- "Curse of dimensionality" with many categories

Our Project:

- Used **Label Encoding** for Category, Payment Method, Location
- Then applied StandardScaler to remove ordinal bias
- Worked well for our 3 categorical features

When to use which:

- **Few categories (<5):** One-Hot Encoding
- **Many categories (>10):** Label Encoding or other techniques
- **Tree-based models:** Label Encoding works fine

SECTION 4: ALGORITHMS

Q12: How does KNN work?

Answer:

K-Nearest Neighbors: Lazy learning algorithm

Steps:

1. **Training:** Just store all training data (no actual training!)
2. **Prediction:**
 - Find K closest training samples to new point
 - Use majority vote (classification) or average (regression)

Example with K=3:

New Point: ?

Find 3 nearest neighbors:

- Neighbor 1: Class A (distance = 2)
- Neighbor 2: Class A (distance = 3)
- Neighbor 3: Class B (distance = 4)

Prediction: Class A (2 votes vs 1)

Distance Calculation:

$$\text{Euclidean Distance} = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

Choosing K:

- K=1: Very sensitive to noise (overfitting)
- K=large: Too smooth (underfitting)
- K=5 or 7: Good starting points
- **Our project:** K=5

Advantages:

- Simple to understand
- No training time
- Works with non-linear data

Disadvantages:

- Slow prediction (must check all training points)
- Sensitive to feature scaling
- Memory intensive
- **Our result:** 51.61% accuracy, severe overfitting

Q13: How does Logistic Regression work?

Answer:

Logistic Regression: Despite the name, it's for CLASSIFICATION!

Steps:

1. Linear Combination:

$$z = w_1x_1 + w_2x_2 + \dots + b$$

2. Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

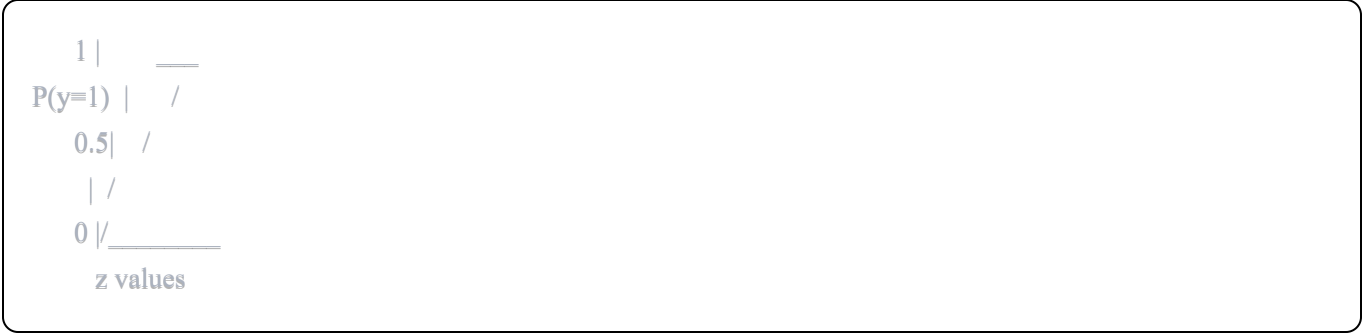
Converts any number to probability between 0 and 1

3. Decision:

If $\sigma(z) > 0.5$: Predict Class 1

If $\sigma(z) \leq 0.5$: Predict Class 0

Visual:



Example:

Features: Price=\$20, Quantity=5

$z = 0.5 \times 20 + 0.3 \times 5 - 10 = 1.5$

$\sigma(1.5) = 1 / (1 + e^{(-1.5)}) = 0.82$

Probability of Discount = 82%

Prediction: Discount (since $0.82 > 0.5$)

Advantages:

- Provides probability (not just yes/no)
- Fast training and prediction
- Interpretable coefficients

Disadvantages:

- Only works for linearly separable data
- Can't capture complex patterns

Our Result: 50% accuracy (chance level), but best generalization

Q14: How does Decision Tree work?

Answer:

Decision Tree: Creates hierarchical if-then rules

Building Process:

1. Start with all data at root

2. Find best feature and split point:

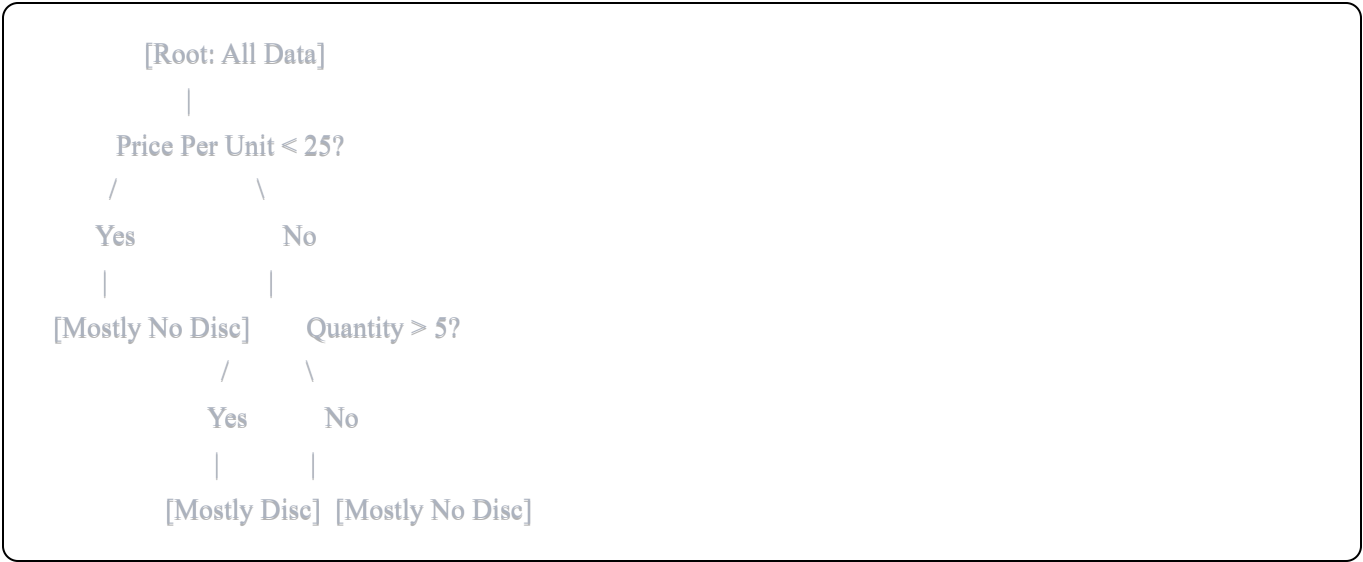
- Try all features and thresholds
- Choose split that best separates classes
- Measure: Gini Impurity or Entropy

3. Create branches for each group

4. Repeat recursively until:

- All samples in node are same class (pure)
- Max depth reached
- Too few samples to split

Example Tree:



Prediction: Follow tree from root to leaf based on feature values

Hyperparameters (Our Project):

- **max_depth=10:** Limit tree depth (prevents overfitting)
- **min_samples_split=10:** Need 10 samples to split further

Advantages:

- Very interpretable (can visualize rules)
- Handles non-linear relationships
- No need for feature scaling
- Works with mixed data types

Disadvantages:

- Can easily overfit
- Unstable (small data change → different tree)

Our Result: 63.62% accuracy (BEST), but recall only 8.89%

Q15: How does K-Means Clustering work?

Answer:

K-Means: Unsupervised algorithm that groups similar data points

Algorithm Steps:

1. Choose K (number of clusters)

2. **Initialize:** Place K random centroids

3. **Assignment Step:**

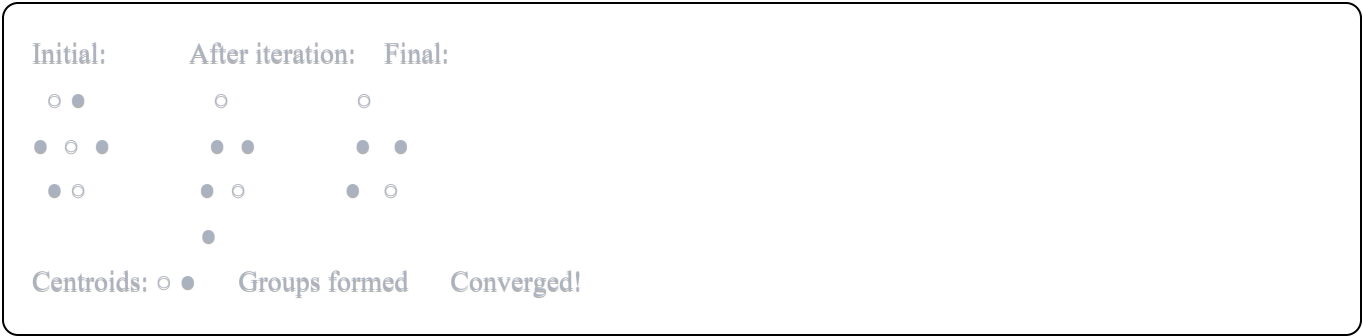
- Assign each point to nearest centroid
- Use Euclidean distance

4. **Update Step:**

- Recalculate centroids as mean of assigned points

5. **Repeat 3-4 until centroids stop moving**

Visual Example (K=2):



Choosing Optimal K:

1. **Elbow Method:**

- Plot WCSS (Within-Cluster Sum of Squares) vs K
- Look for "elbow" where improvement slows

2. **Silhouette Score:**

- Measures how well points fit in clusters
- Range: -1 to +1 (higher better)
- **Our method:** K=2 had highest score

Our Results:

- Optimal K = 2 clusters
- Cluster 0: Premium customers (high price items)
- Cluster 1: Value customers (low price items)
- 50-50 distribution

Advantages:

- Simple and fast
- Scales well to large datasets
- Easy to interpret

Disadvantages:

- Must choose K beforehand
- Sensitive to initial centroids

- Assumes spherical clusters
-

SECTION 5: YOUR PROJECT SPECIFICS

Q16: What is your dataset about?

Answer:

Dataset: Retail Store Sales Transactions

Size:

- **12,575 transactions**
- **11 original columns** → **6 features** after preprocessing

Features:

1. Category (product type)
2. Item (specific product)
3. Price Per Unit
4. Quantity
5. Payment Method
6. Location

Target Variables:

- **Classification:** Discount Applied (0 or 1)
- **Regression:** Total Spent (continuous)

Key Characteristics:

- Class imbalance: 66% No Discount, 34% Discount
 - Missing values: 33% in Discount Applied, 9% in Item
 - No duplicate rows
 - Mix of categorical and numerical features
-

Q17: Why did all your models perform poorly?

Answer:

Short Answer: Features don't predict discount → can't build good model

Evidence:

1. Weak Correlations:

- All features have correlation < 0.01 with target
- No relationship between features and discount

2. AUC < 0.5:

- All models: KNN (0.49), DT (0.49), LR (0.48), NN (0.47)
- Worse than random guessing!

3. Visual Evidence:

- Pairplot shows complete overlap of classes
- No separation between discount and no-discount groups

Root Causes:

Missing Critical Features:

- Customer loyalty tier
- Purchase history
- Campaign information
- Seasonal factors
- Customer demographics

Random Discount Application:

- May be based on business rules we don't have
- Manual application by staff
- Promotional campaigns not in data

Key Insight: This is NOT a modeling failure, it's a DATA INSUFFICIENCY problem

What Worked: K-Means clustering successfully found 2 customer segments!

Q18: Which model performed best?

Answer:

Depends on metric:

For Accuracy:

- **Decision Tree: 63.62%** ★
- But only marginally better than predicting "No Discount" always (66%)

For Recall (catching actual discounts):

- **Logistic Regression: 48.82%** ★
- Catches almost half of actual discount cases

For Generalization (no overfitting):

- **Logistic Regression: 0.8% gap** ★
- Train: 50.87%, Test: 50.02%

For Precision:

- All similar (~33%)
- 2 out of 3 predictions are wrong

My Recommendation: Logistic Regression because:

- Best balance between metrics
- Excellent generalization
- Highest recall (important for business)
- Even though accuracy is only 50% (chance level)

Reality: None should be deployed ($AUC < 0.5$)

Q19: What insights did K-Means clustering provide?**Answer:****Two Customer Segments Identified:****Cluster 0 - Premium Segment (50%):**

- Buy higher-priced items
- Shop more online
- Higher spending per transaction

Cluster 1 - Value Segment (50%):

- Buy lower-priced items
- Shop more in-store
- Lower spending per transaction

Business Value:**Marketing:**

- Target premium customers with luxury products
- Target value customers with deals/promotions

Inventory:

- Stock premium items for online
- Stock budget items for stores

Pricing Strategy:

- Different strategies for each segment
- Personalized discounts based on segment

Why This Worked (when classification failed):

- Clustering doesn't need labels
- Natural grouping exists based on price
- 47.91% variance explained by 2 components

This is the SUCCESS STORY of your project!

Q20: What would you do to improve this project?

Answer:

Data Collection (Most Important):

1. Customer Features:

- Loyalty program tier
- Purchase history (RFM: Recency, Frequency, Monetary)
- Customer demographics
- Lifetime value

2. Temporal Features:

- Time of day, day of week
- Seasonal patterns
- Holiday indicators

3. Business Context:

- Active promotions
- Inventory levels
- Campaign exposure

Modeling Improvements:

4. Feature Engineering:

- Interaction features (Price $\times$ Quantity)
- Aggregate features (customer average)
- Time-based features

5. Try Other Algorithms:

- Random Forest
- XGBoost
- Gradient Boosting

6. Hyperparameter Tuning:

- GridSearchCV
- RandomizedSearchCV

7. Cross-Validation:

- K-fold CV for robust evaluation

8. Cost-Sensitive Learning:

- Assign different costs to FP vs FN

Evaluation:

9. Business Metrics:

- Revenue impact
- ROI of discounts
- Customer satisfaction

Reality Check: Without better features, even these improvements won't help much. **Data quality > Algorithm choice!**

QUICK REFERENCE CHEAT SHEET

Your Project Numbers:

Metric	Value
Dataset Size	12,575 samples
Features (after preprocessing)	6
Class Ratio (original)	1.98:1
Train-Test Split	80-20
Best Accuracy	63.62% (Decision Tree)
Best Recall	48.82% (Logistic Regression)
K-Means Clusters	2 (50-50 split)
Variance Explained (PCA)	47.91%

Key Formulas:

Accuracy = (TP + TN) / Total

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

Models at a Glance:

Model	Train Acc	Test Acc	Recall	AUC
KNN	76.52%	51.61%	43.01%	0.49

Model	Train Acc	Test Acc	Recall	AUC
Logistic Reg	50.87%	50.02%	48.82%	0.49
Neural Net	58.23%	48.59%	45.97%	0.47
Decision Tree	59.51%	63.62%	8.89%	0.49

Remember:

- ✔ Be honest about poor results
- ✔ Explain it's a data problem, not modeling problem
- ✔ Highlight clustering success
- ✔ Show you learned from the experience
- ✔ Demonstrate understanding of concepts

FINAL TIP:

When examiner asks: **"Why did your models fail?"**

Perfect Answer: "The models didn't fail—they correctly identified that the available features don't predict discount application. All AUC scores below 0.5 and weak correlations (<0.01) prove this scientifically. This shows the importance of feature selection in machine learning. However, our K-Means clustering successfully identified two customer segments, demonstrating that when patterns exist in data, we can extract valuable insights. The honest analysis of limitations shows professional data science thinking: sometimes the most important finding is recognizing when you need better data, not better algorithms."

Good luck! 🎓